Milestone Compliance Report 4
Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming

Abstract:

This report will describe our progress towards our first prototype for our capstone project. We begin by briefly describing specific technical elements of our game at a high level, followed by a walkthrough of the approaches we are currently investigating after our initial profiling results. We then describe our tentative testing methodology and setup, followed by a more in-depth analysis of the dispatched wavefronts in our current code, and outline of our profiling and benchmark tools, and conclude with our next steps and remaining tasks from this cycle.

CPU-GPU Workload Split

At the core of our project are seven kernels which are dispatched and run for each frame to simulate the fluid in our game. The seven kernels are: *ExternalForces* (currently just gravity), *SpatialHash* (pt 1 of our broad-phase spatial subdivision algorithm), *SortAndCalculateOffsets* (pt 2 of spatial subdivision), *CalculateDensities*, *CalculatePressureForce*, *CalculateViscosity*, and *UpdatePositions*. Each of the density, pressure, and viscosity kernels index the results of our broad-phase collision detection to generate properties for each particle within the radius of its neighbors. Important elements to note are that particles have true data dependencies between stages, and as a result, threads waiting to execute a given calculation must wait for the calculation before it (density feeds into pressure force which is fed into viscosity); and that the CPU-GPU boundary will be complicated due to the need to merge particle data at boundary points.

Based on early profiling analyses, the combination of our density, pressure, and viscosity calculations take up 81% of our processor time, without accounting for the slowness of our current sorting algorithm (we currently use bitonic merge sort, we expect closer to 90% with better sorting). Each of the kernels performs a narrow phase distance check before performing a series of vector calculations based on the positions and momentums of its neighbors, making it the primary bottleneck and our target to improve with CPU-GPU workload splitting. To avoid unnecessary data transfers, we need to be able to split our workload by density without much communication - which is quite difficult to do without simply passing an entire buffer between calculations for each frame.

We are currently working on two methods for splitting the target workload by density, both compatible with the density-based subdivision approach described in the last MCR. The first method involves pushing the result of each density calculation to a lock-free stack or queue, with some quantity of subsequent pressure and viscosity calculations being performed by the CPU. The idea is that the density calculations which evaluate the fastest will generally be the regions of lower particle density, and so pushing the results of each region to a stack/queue

should heuristically sort the regions by decreasing/increasing densities respectively. Our second method involves adding an additional step to our density kernel which saves a copy of the final density calculations for use in our next frame. At the cost of extra memory, we can save to a hash the density values of our previous frame before we dispatch into our next frame, however precision will be limited and fast-moving particles may not be tracked accurately.

In theory, the CPU may be able to evaluate extreme regions (highest or lowest densities) faster than the GPU, however our current theory of which will actually be faster has changed since analyzing the thread profiling for our application (see “Preliminary Wavefront Analysis of GPU-Only Implementation”).

Lastly, we note that the early prototype will statically dispatch to the CPU a fixed quantity of regions, as dynamic dispatching based on workload may be established after we are able to see some sort of improvement.

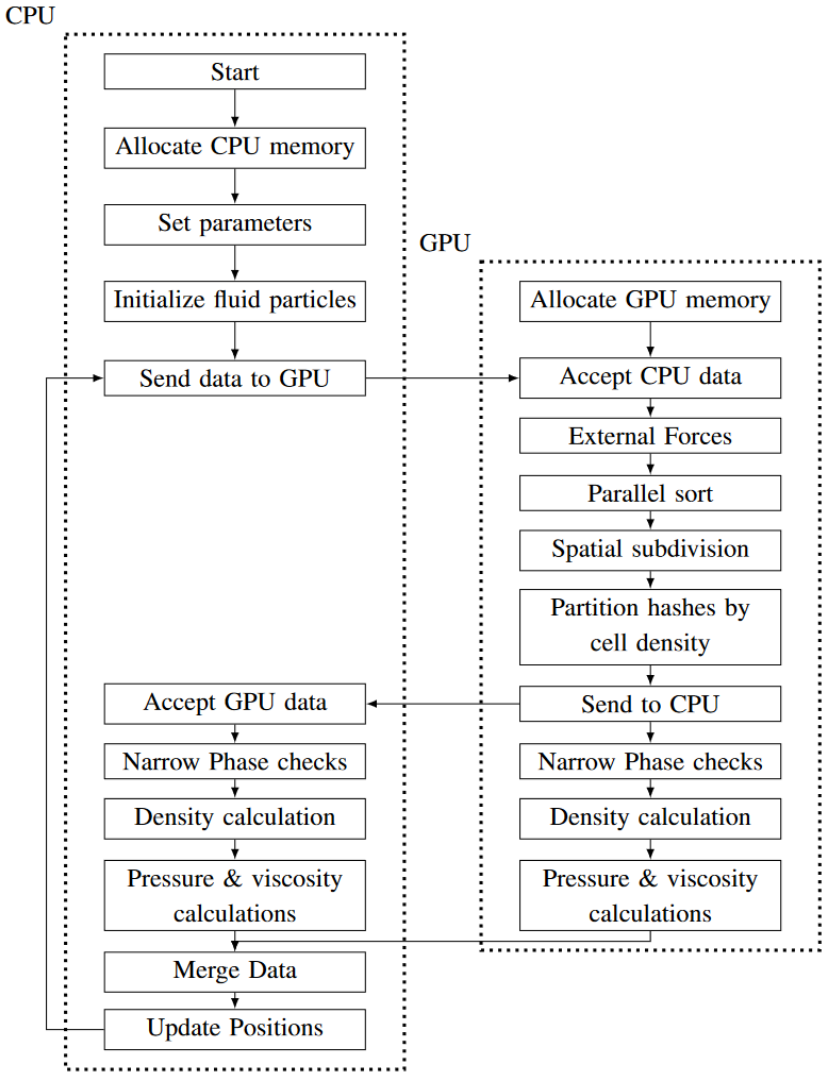

Figure 1: *Updated density-based subdivision flowchart.*

Testing Methodology and Setup

We have developed a testing methodology to allow us to evaluate the performance of both the CPU-GPU collision implementation (which is currently being developed) as well as the GPU only baseline. The testing setup involves two programs, one which evaluates the overall performance over a period of time, as well as an in depth single frame evaluation process using third party profiling tools. The overall performance of the implementation is evaluated as the average CPU / GPU frame time over 10 seconds. This average is obtained for at least three different particle counts (ex. 1000, 5000, and 10000) which gives multiple data points that can be used to investigate bottlenecks and work towards further improvement enhancements. The performance during a single frame will also be collected using a third party GPU profiling tool (either Radeon Developer Suite or Nvidia NSight). This data will allow us to determine where the bottlenecks for our algorithm are by giving an insight into the wavefronts/waves being executed. The goal for the CPU-GPU implementation is to reduce the frame time of the simulation. The simulations used for testing will resemble the figure below.

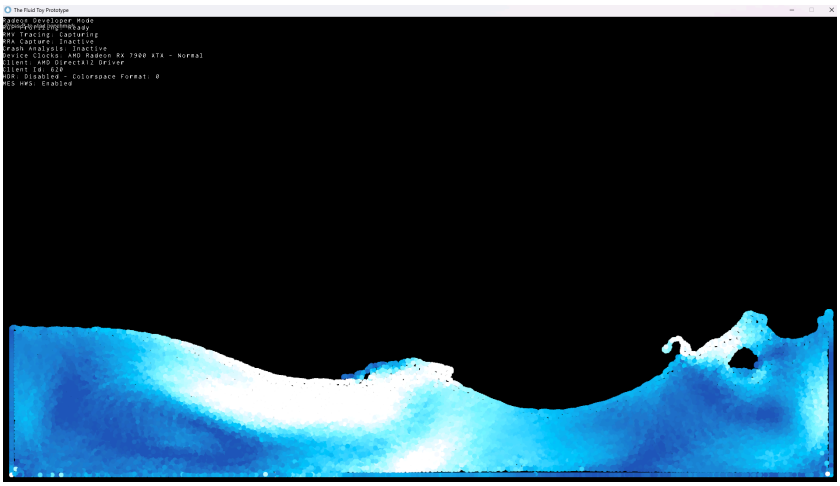

***Figure 2**: Particle Simulation Testbench*

Preliminary Wavefront Analysis of GPU-Only Implementation

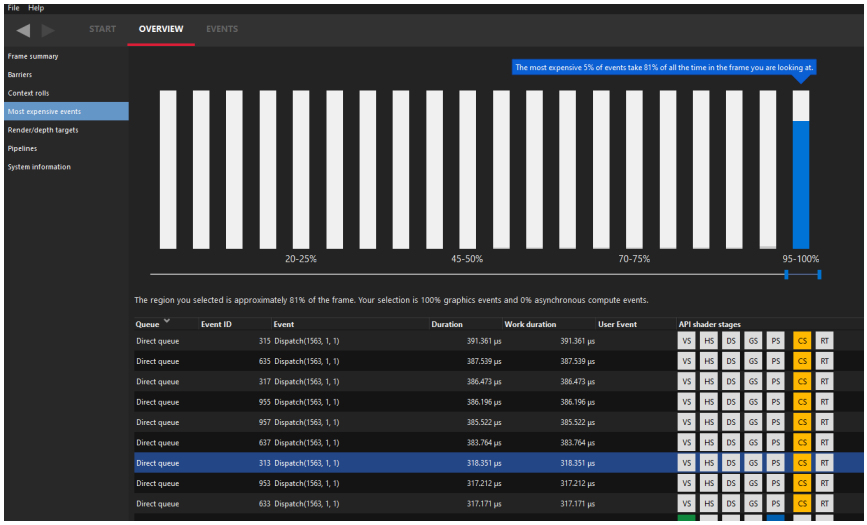

***Figure 3**: Most Expensive events from the* Radeon Developer Suite *GPU profiler*

The "big three" dispatch calls of our application were density, pressure, and viscosity calculations. In fourth place was the GPU sorting algorithm, but that is more difficult to optimize.

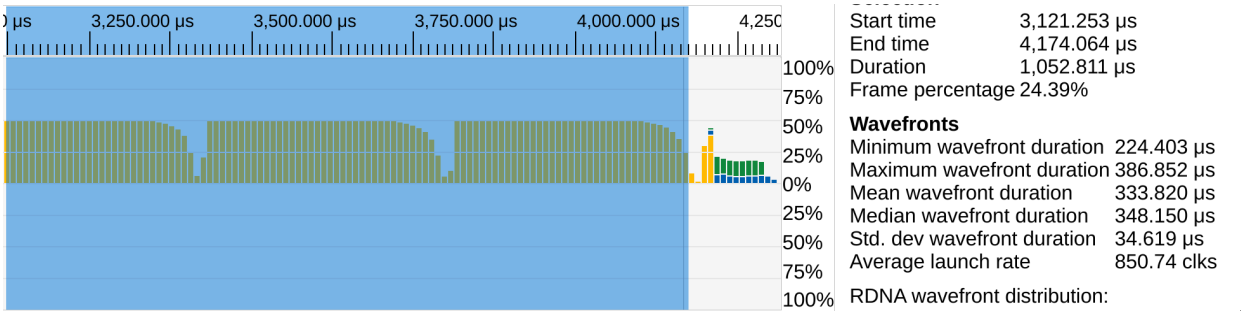

***Figure 4:** GPU utilization for density, pressure, and viscosity dispatches*

We found that there is a significant gap between minimum wavefront duration and maximum wavefront duration for the big three. We call this gap the "long tail", as most of the work in a dispatch call is parallel, and then the work begins to trail off as different wavefronts exit as the workload becomes sequential. As an example, 29% of the density dispatch is spent on this long tail.

The reason why the long tail is important is because all calculations must be done before we can move to the next dispatch call. As an example, even if 99% of the density computation is done, we cannot move onto the pressure computation until the 1% remaining is complete, so the long tail determines the performance of our application.

We speculate that this long tail occurs primarily because of high density areas in the fluid simulation. Moving this high-density area to the CPU would result in better performance than using the CPU or GPU on their own as it would reduce the length of that long tail.

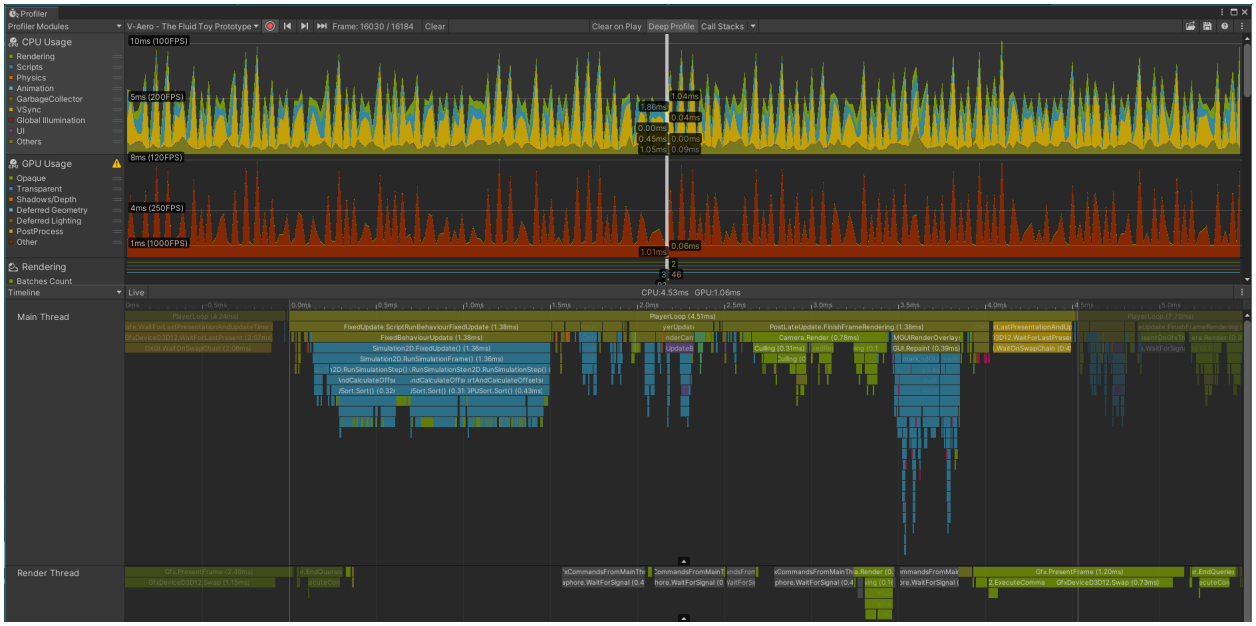

Figure 5: *Timeline of Frame time data showing variations between frames*

This is one of the built in profiling tools that Unity provides, it allows us to see variations in frame data and the corresponding calls that may be causing those variations, allowing us to target those areas for optimization. One of the advantages of using the built in tools provided by Unity is that they work consistently between hardware configurations from Nvidia and AMD and can even be used to profile other platforms like Android.

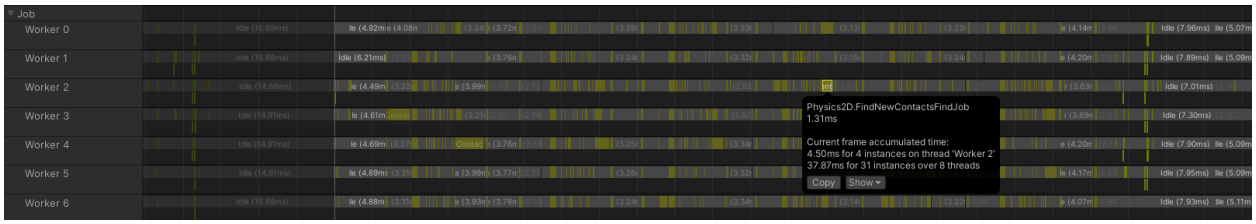

Figure 6: *Snapshot of CPU Thread Usage for a Frame*

This breakdown is also a part of Unity's built in profiling suite. It provides a timeline of CPU usage and a breakdown of the specific calls handled by each thread. This will allow us to ensure that we are efficiently distributing the workload between threads for our simulation.

Another note worth mentioning that we may use in the future as our work expands into other platforms is the Android GPU Inspector which is a profiling tool built specifically for Android.

Figure 7: *Generic Screenshot showing the capabilities of Android GPU Inspector*

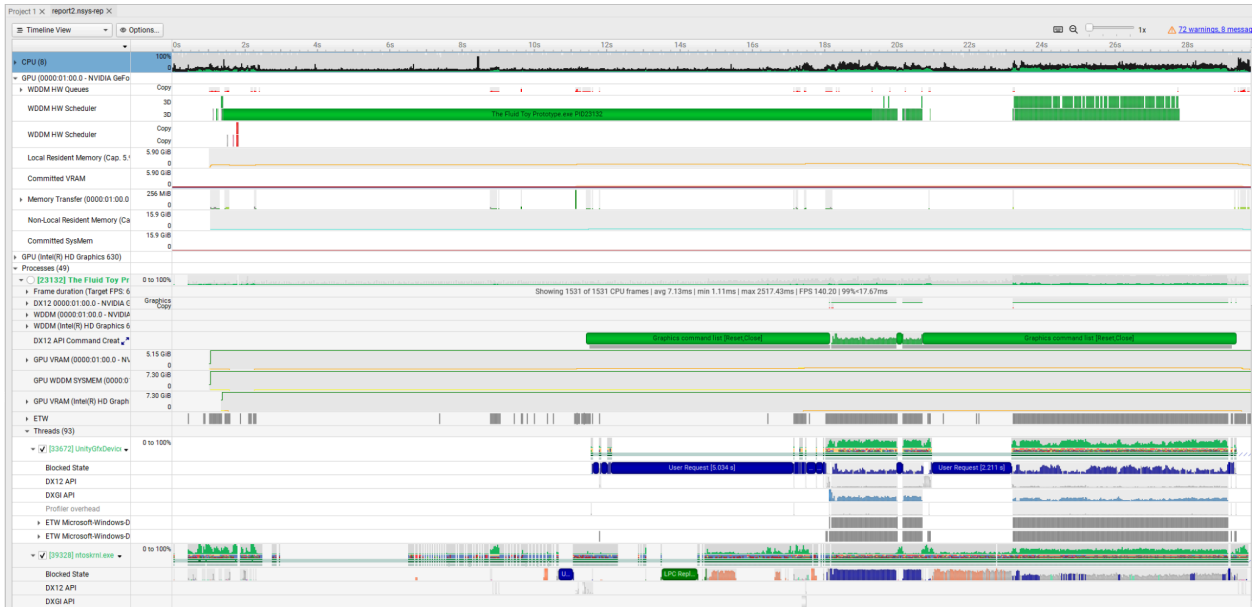

Figure 8: *Screenshot showing Nvidia Nsight*

Similar to the Radeon Developer Suite, Nvidia Nsight and Nsight Compute give us statistics about wave coherency, active threads, warps and thread blocks, memory operations, kernel code, etc but for Nvidia GPUs. We mostly used the Radeon Developer Suite for this period.

Conclusion:

In this milestone, we have made significant strides toward a functional prototype for our capstone project. Our work has focused on refining and profiling core components, particularly the kernels central to our fluid simulation—density, pressure, and viscosity calculations—which represent the majority of our processing workload. Early profiling results have led to promising new directions in our CPU-GPU workload split, aimed at mitigating performance bottlenecks by reducing long-tail wavefront delays and optimizing data dependencies across the CPU-GPU boundary.

We have established a rigorous testing methodology and setup, which enables us to benchmark both the current GPU-only implementation and our developing CPU-GPU approach. Through multiple levels of profiling, we can closely observe the computational demands of each kernel and iteratively target bottlenecks. Additionally, our initial wavefront analysis has provided insights that will guide us in achieving a more efficient dispatch strategy, ultimately aimed at reducing frame times and improving real-time performance.

Looking ahead, our next steps involve refining the workload split, as well as further testing across diverse particle counts and density configurations. As we continue, we anticipate incorporating dynamic dispatch and testing new data transfer strategies, guided by insights from our profiling data. This progress establishes a strong foundation for the continued development of our capstone project and positions us well for future optimizations.

-